Contents

1. INTRODUCTION AND PROJECT OVERVIEW 3
1.1 Project Context 3
1.2 Structural Patterns Overview 4
1.3 Deliverable Requirements Met 4
2. BRIDGE PATTERN CASES 5
2.1 BR1 — Bridge Pattern: Notification System 5
2.1.1 Problem Statement 5
2.1.2 Solution Design 5
2.1.3 Implementation Details 6
2.1.4 Why Bridge Pattern? 7
2.1.5 Demo Execution 7
2.2 BR2 — Bridge Pattern: Report Content × Renderer 8
2.2.1 Problem Statement 8
2.2.2 Solution Design 9
2.2.3 Implementation Details 9
2.2.4 Why Bridge Pattern? 10
2.2.5 Demo Execution 10
3. ADAPTER PATTERN CASES 11
3.1 AD1 — Adapter Pattern: External Patient System 11
3.1.1 Problem Statement 11
3.1.2 Solution Design 11
3.1.3 Implementation Details 12
3.1.4 Why Adapter Pattern? 13
3.1.5 Demo Execution 13
3.2 AD2 — Adapter Pattern: Payment Gateway 14
3.2.1 Problem Statement 14
3.2.2 Solution Design 14
3.2.3 Implementation Details 15
3.2.4 Why Adapter Pattern? 16

3.2.5 Demo Execution 16
4. FLYWEIGHT PATTERN CASE 17
4.1 FW1 — Flyweight Pattern: Medical Code Catalog 17
4.1.1 Problem Statement 17
4.1.2 Solution Design 17
4.1.3 Implementation Details 18
4.1.4 Why Flyweight Pattern? 19
4.1.5 Demo Execution 19
5. DECORATOR PATTERN CASES 19
5.1 DC1 — Decorator Pattern: Report Enhancements 19
5.1.1 Problem Statement 19
5.1.2 Solution Design 20
5.1.3 Implementation Details 20
5.1.4 Why Decorator Pattern? 21
5.1.5 Demo Execution 22
5.2 DC2 — Decorator Pattern: Notification Sender 22
5.2.1 Problem Statement 22
5.2.2 Solution Design 23
5.2.3 Implementation Details 23
5.2.4 Why Decorator Pattern? 24
5.2.5 Demo Execution 25
6. DEVOPS AND PROJECT MANAGEMENT 25
6.1 Maven Configuration 25
6.2 Git Repository Structure 26
6.3 JIRA Project Management 26
7. ASSUMPTIONS AND DESIGN DECISIONS 29
7.1 Assumptions Made 29
7.2 Design Decisions 29
8. CONCLUSION 30
8.1 Summary of Deliverable #2 30
8.2 Lessons Learned 30

8.3 Future Enhancements 31

1. INTRODUCTION AND PROJECT OVERVIEW

1.1 Project Context

The Hospital Management System (HMS) is a comprehensive software solution designed to manage patient records, appointments, billing, medical records, and notifications. In Deliverable #2, we implement seven structural design pattern cases to improve system flexibility, maintainability, and scalability.

1.2 Structural Patterns Overview

Structural design patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient. For this deliverable, we implement:

2 Bridge Pattern cases: Decouple abstraction from implementation

2 Adapter Pattern cases: Convert incompatible interfaces

1 Flyweight Pattern case: Share objects to support large numbers

2 Decorator Pattern cases: Add responsibilities to objects dynamically

1.3 Deliverable Requirements Met

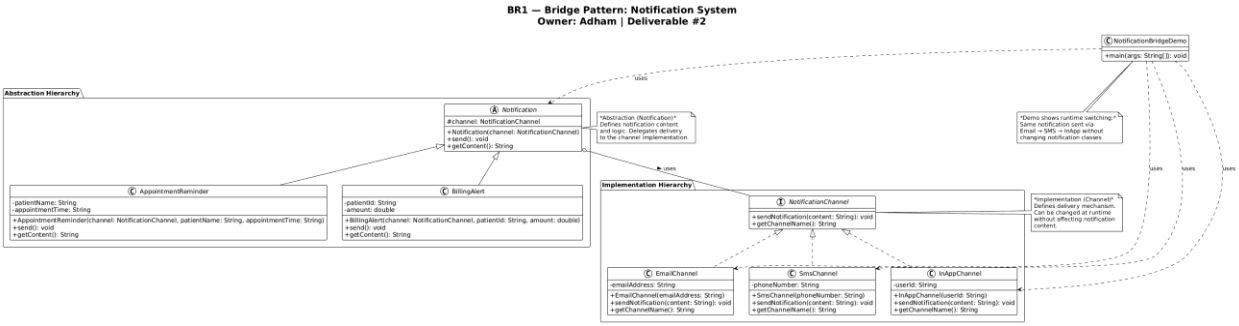
Requirement	Status	Evidence
2 Bridge cases implemented	☑ Complete	BR1, BR2 UML + Java + Demo
2 Adapter cases implemented	☑ Complete	AD1, AD2 UML + Java + Demo
1 Flyweight case implemented	☑ Complete	FW1 UML + Java + Demo
2 Decorator cases implemented	☑ Complete	DC1, DC2 UML + Java + Demo
Maven project builds	☑ Complete	`mvn clean package` SUCCESS
All demos run without errors	☑ Complete	Menu options 8-14 tested
JIRA timeline with dependencies	☑ Complete	Screenshots in Section 6

---

2. BRIDGE PATTERN CASES

2.1 BR1 — Bridge Pattern: Notification System

2.1.1 Problem Statement



The Hospital Management System needs to send various types of notifications (appointment reminders, billing alerts, lab results) through multiple channels (Email, SMS, In-App, Push). Without the Bridge pattern, we would need to create a class for every combination:

AppointmentReminderEmail

AppointmentReminderSMS

AppointmentReminderInApp

BillingAlertEmail

BillingAlertSMS

... (12+ classes!)

This creates a class explosion problem and makes it difficult to add new notification types or channels without modifying existing code.

2.1.2 Solution Design

We apply the Bridge pattern to separate notification **content** (abstraction) from notification **delivery channel** (implementation).

Abstraction Hierarchy:

`Notification` (abstract) — Base for all notification types

`AppointmentReminder` — Appointment-specific content

`BillingAlert` — Billing-specific content

`LabResultNotification` — Lab result content

Implementation Hierarchy:

`NotificationChannel` (interface) — Delivery mechanism

`EmailChannel` — Email delivery

`SmsChannel` — SMS delivery

`InAppChannel` — In-application notification

`PushNotificationChannel` — Push notification

The Bridge:
The `Notification` class holds a reference to `NotificationChannel`, allowing any notification type to use any channel at runtime.

2.1.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.bridge.notification`

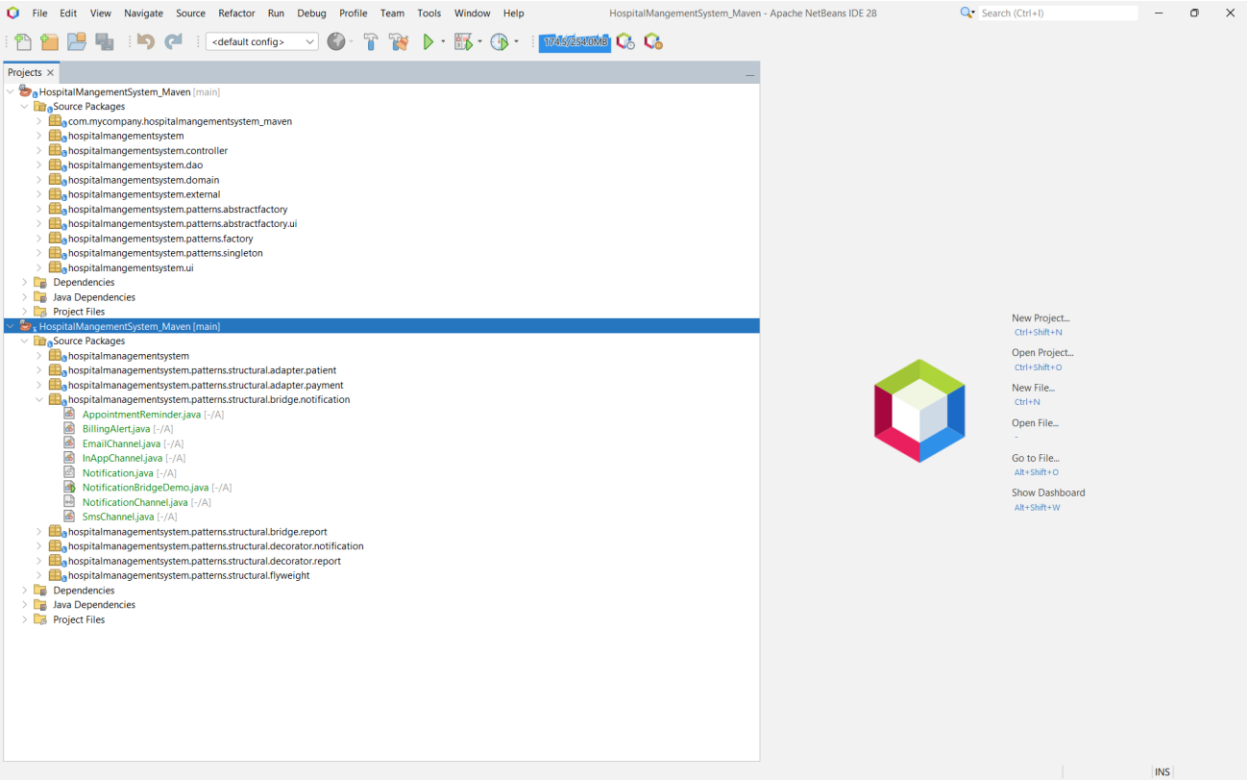
Key Classes:

```
// Abstraction
public abstract class Notification {
    protected NotificationChannel channel;
    public Notification(NotificationChannel channel) { this.channel =
channel; }
    public abstract String generateMessage();
    public void send() { channel.sendNotification(generateMessage()); }
}

// Refined Abstraction
public class AppointmentReminder extends Notification {
    private String patientName, appointmentTime, doctorName;
    @Override
    public String generateMessage() {
        return "Hello " + patientName + ", you have an appointment with Dr. "
               + doctorName + " at " + appointmentTime;
    }
}

// Implementation
public interface NotificationChannel {
    void sendNotification(String content);
    String getChannelName();
```

}



2.1.4 Why Bridge Pattern?

Without Bridge	**With Bridge**
12+ classes for all combinations	7 classes (3 notifications + 4 channels)
Adding SMS requires 3 new classes	Adding SMS requires 1 new class
Adding new notification type requires 4 new classes	Adding notification type requires 1 new class
Tight coupling between content and delivery	Complete separation of concerns

Benefits Achieved:

Runtime Flexibility: Same appointment reminder sent via Email for records, SMS for urgency

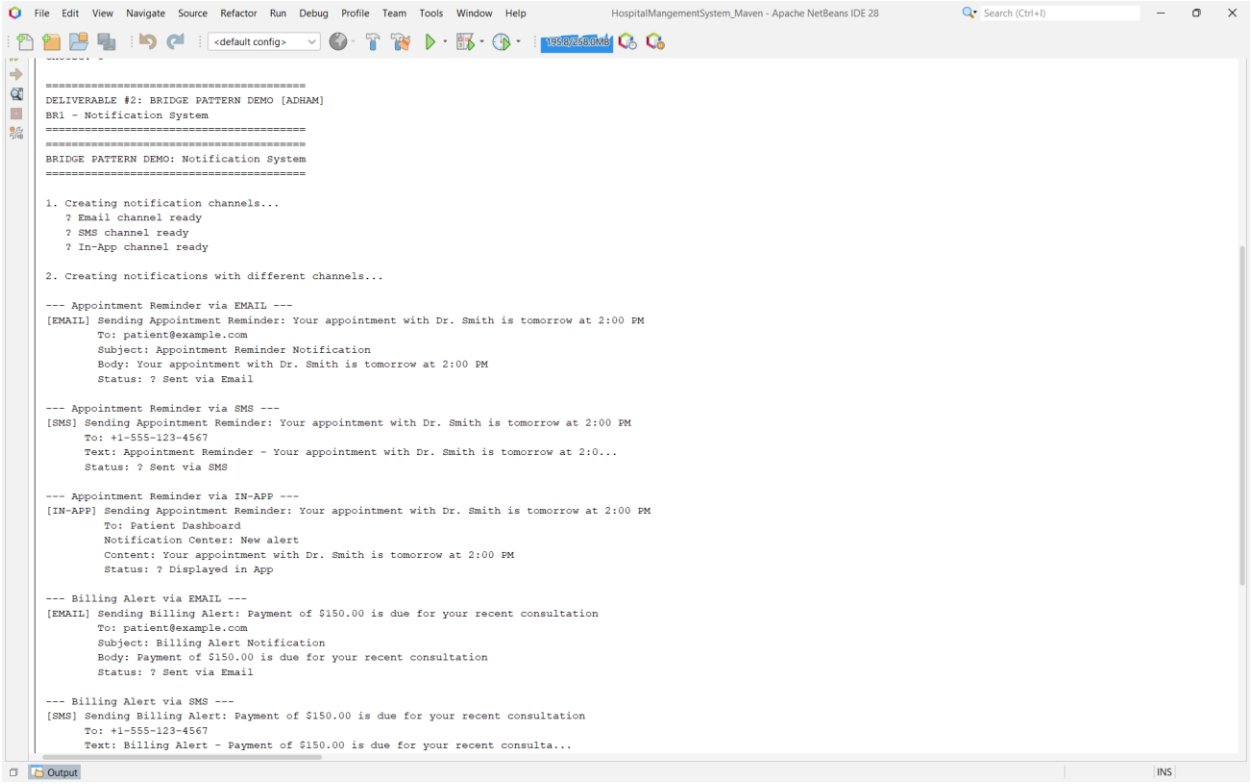
Independent Evolution: Add WhatsApp channel without touching notification classes

Single Responsibility: Notification classes handle content, Channels handle delivery

Open/Closed Principle: Extend without modifying existing code

2.1.5 Demo Execution

Menu Option: 8 — Test Notification System (Bridge Pattern Demo)



Demo Scenarios:

AppointmentReminder → EmailChannel (professional record)

AppointmentReminder → SMSChannel (immediate alert)

BillingAlert → InAppChannel (user preference)

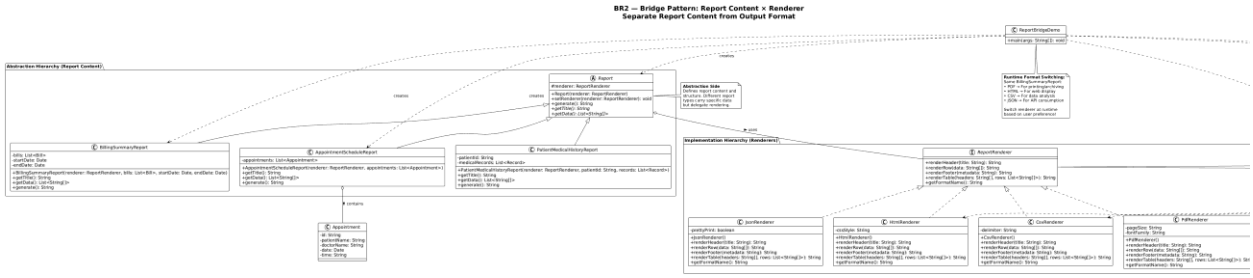
LabResultNotification → PushNotificationChannel (urgent)

Runtime switching: Change channel without recreating notification

---

2.2 BR2 — Bridge Pattern: Report Content × Renderer

2.2.1 Problem Statement



The HMS generates various reports (appointment schedules, billing summaries, patient histories)

that need to be output in multiple formats (PDF for printing, HTML for web, CSV for analysis, JSON for APIs). Without the Bridge pattern, each report would need separate implementations for each format, leading to code duplication and maintenance nightmare.

2.2.2 Solution Design

We separate report **content** from report **rendering format**.

Abstraction Hierarchy:

`Report` (abstract) — Base for all report types

`AppointmentScheduleReport` — Appointment data

`BillingSummaryReport` — Billing data

`PatientMedicalHistoryReport` — Medical history data

Implementation Hierarchy:

`ReportRenderer` (interface) — Output format

`PdfRenderer` — PDF output

`HtmlRenderer` — HTML output

`CsvRenderer` — CSV output

`JsonRenderer` — JSON output

The Bridge:
Reports delegate rendering to the renderer interface, allowing format switching at runtime.

2.2.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.bridge.report`

Key Design:

```
public abstract class Report {
    protected ReportRenderer renderer;
    public void setRenderer(ReportRenderer renderer) { this.renderer =
renderer; }
    public abstract String getTitle();
    public abstract List<String[]> getData();

    public String generate() {
        return renderer.renderHeader(getTitle())
             + renderer.renderTable(getHeaders(), getData())
             + renderer.renderFooter("Generated by HMS");
    }
```

}

```
========================================
DELIVERABLE #2: BRIDGE PATTERN DEMO [ANDREW]
BR2 - Report (Content x Renderer)
========================================
=== Bridge Pattern: Report (Content x Renderer) Demo ===
Author: [ANDREW]

--- Demo 1: Appointment Schedule Report ---

[PDF format]
╔═══════════════════════════════════════════╗
║  PDF REPORT                               ╒
╒  Title: AppointmentScheduleReport         ╒
╠═══════════════════════════════════════════╣
╒  Appt ID | Patient NID | Date/Time | Status
╒  A1 | 123 | 2025-12-30 10:00 | SCHEDULED
╒  A2 | 123 | 2026-01-02 12:30 | SCHEDULED
╒  B1 | 999 | 2025-12-28 09:00 | CANCELLED
╒  C1 | 456 | 2026-01-15 14:00 | SCHEDULED
╠═══════════════════════════════════════════╣
╒  [Generated as PDF format]                 ╒
╚═══════════════════════════════════════════╝
```

2.2.4 Why Bridge Pattern?

Benefits:

Format Flexibility: Doctor wants PDF, Analyst wants CSV, System wants JSON — same report, different renderers

Extensible: Add Excel renderer without modifying existing reports

Consistent Interface: All reports use same rendering contract

Testability: Mock renderers for unit testing report logic

2.2.5 Demo Execution

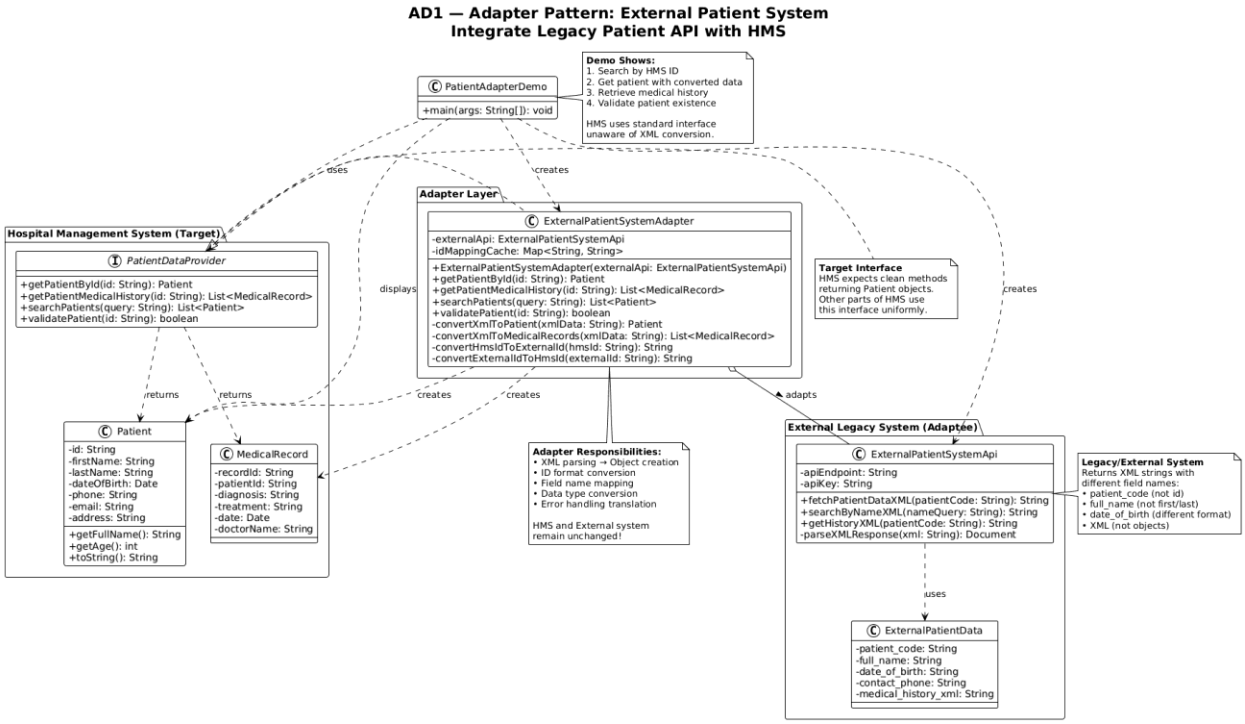
Menu Option: 9 — Test Report Renderer (Bridge Pattern Demo)

---

3. ADAPTER PATTERN CASES

3.1 AD1 — Adapter Pattern: External Patient System

3.1.1 Problem Statement



The HMS needs to integrate with a legacy external patient database system that provides data in XML format with different field names:

External: `patient_code`, `full_name`, `date_of_birth`

HMS: `id`, `firstName`, `lastName`, `dateOfBirth`

The external API returns XML strings, while HMS expects Patient objects. We cannot modify the external system (legacy, third-party) or the HMS interface (used throughout the system).

3.1.2 Solution Design

Create an **Adapter** that converts between external system's interface and HMS expected interface.

Components:

Target (`PatientDataProvider`): Interface HMS expects

Adaptee (`ExternalPatientSystemApi`): Legacy system with XML responses

Adapter (`ExternalPatientSystemAdapter`): Converts XML to Patient objects, translates IDs

Client (HMS modules): Uses Target interface, unaware of adaptation

3.1.3 Implementation Details

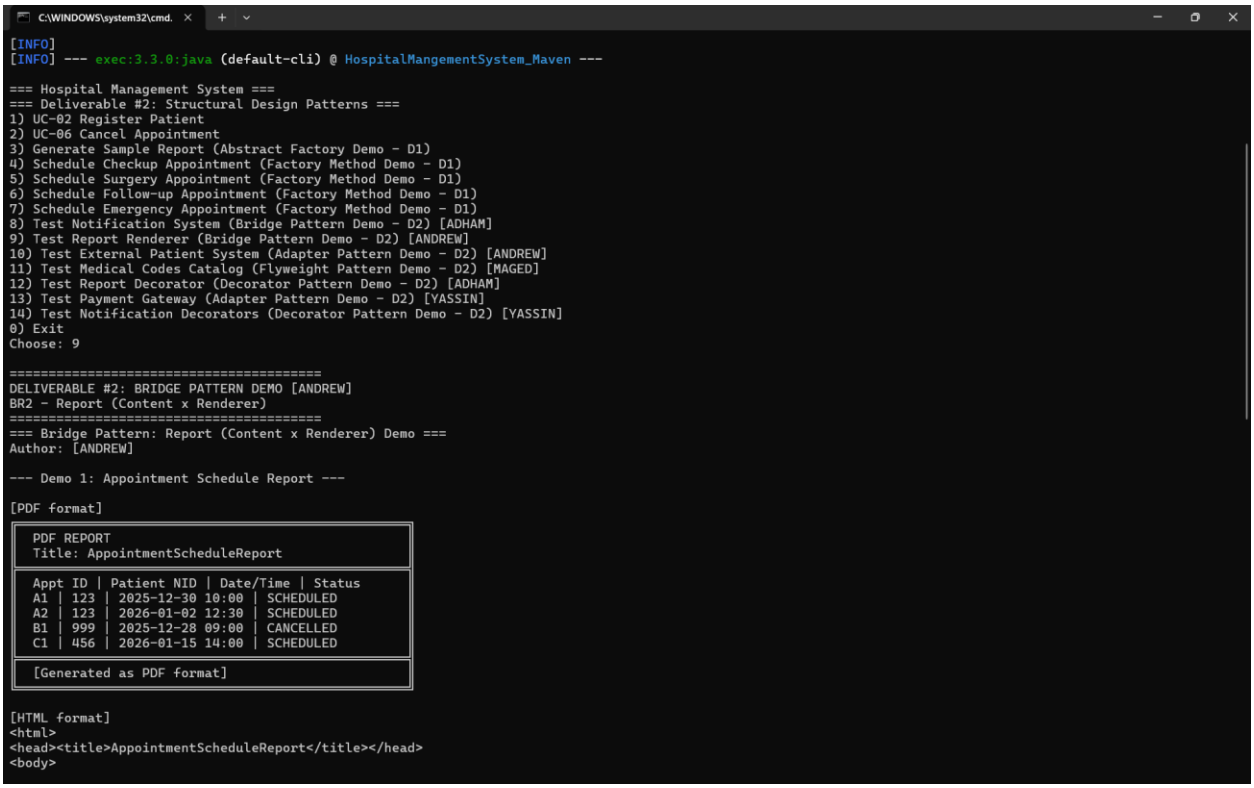
Location: `hospitalmanagementsystem.patterns.structural.adapter.patient`

Adapter Logic:

```
public class ExternalPatientSystemAdapter implements PatientDataProvider {
    private ExternalPatientSystemApi externalApi;

    @Override
    public Patient getPatientById(String id) {
        String externalId = convertHmsIdToExternalId(id);
        String xmlResponse = externalApi.fetchPatientDataXML(externalId);
        return convertXmlToPatient(xmlResponse);
    }

    private Patient convertXmlToPatient(String xml) {
        // Parse XML, extract fields, create Patient object
        // Map: patient_code → id, full_name → firstName/lastName, etc.
    }
}
```



3.1.4 Why Adapter Pattern?

Aspect	Without Adapter	With Adapter
Integration	Modify HMS everywhere	Single adapter class
External Changes	Break HMS code	Update adapter only
Testing	Complex with real external API	Mock adapter for testing
Reusability	Tight coupling	Clean interface for future integrations

Conversions Handled:

XML parsing → Object creation

`patient_code` → `id` mapping

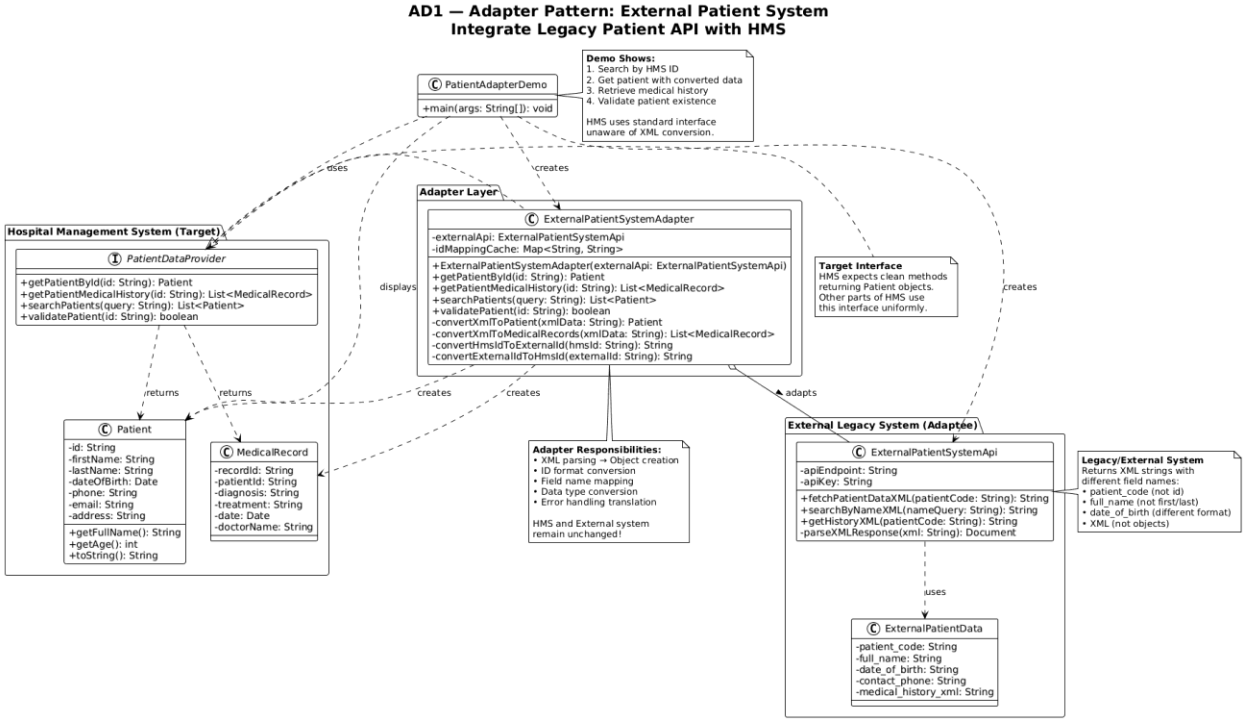
`full_name` → `firstName` + `lastName` splitting

Date format conversion

ID format translation (HMS-P-123 ↔ EXT_12345)

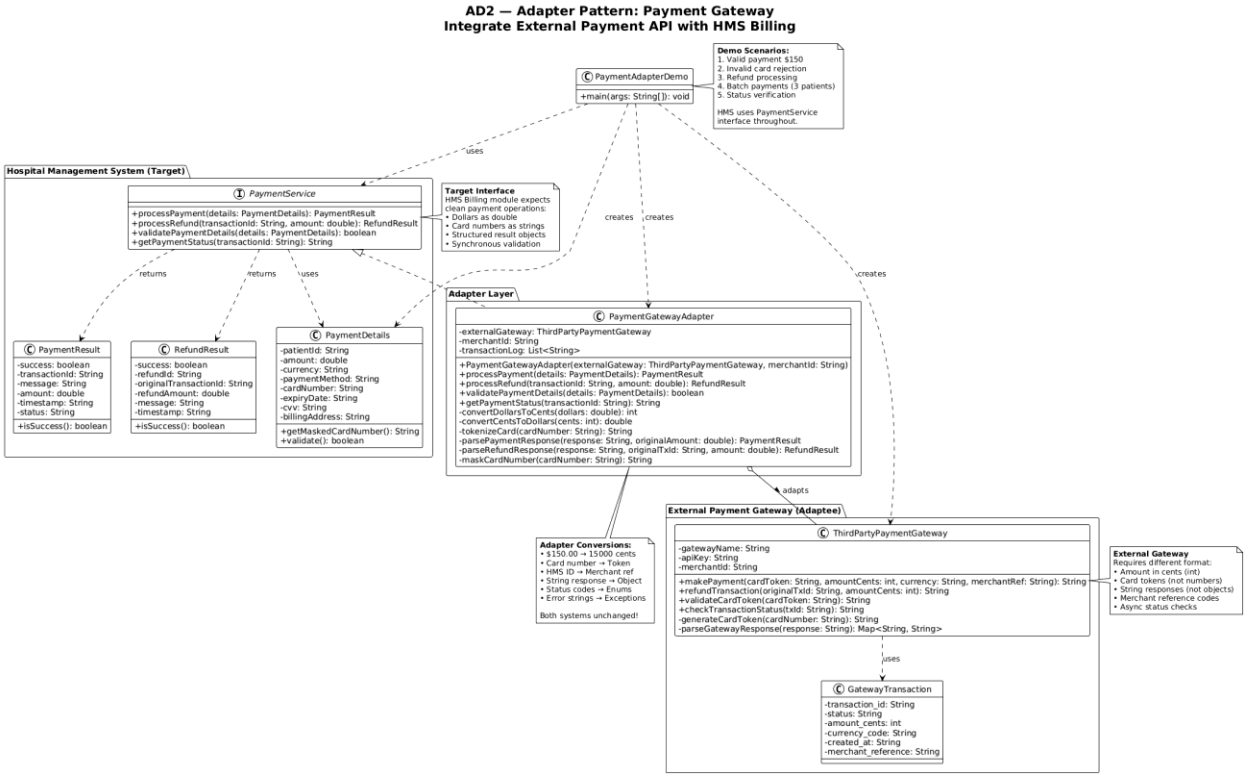
3.1.5 Demo Execution

Menu Option: 10 — Test External Patient System (Adapter Pattern Demo)



3.2 AD2 — Adapter Pattern: Payment Gateway

3.2.1 Problem Statement



HMS Billing module needs to process payments through third-party gateways (Stripe, PayPal, etc.) that use different:

Currency formats (dollars vs cents)

Card handling (numbers vs tokens)

Response formats (strings vs objects)

API structures

The HMS `PaymentService` interface expects clean operations, but external gateways have proprietary formats.

3.2.2 Solution Design

Components:

Target (`PaymentService`): HMS billing interface

Adaptee (`ThirdPartyPaymentGateway`): External payment API

Adapter (`PaymentGatewayAdapter`): Converts HMS format to gateway format

3.2.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.adapter.payment`

Key Conversions:

```
public class PaymentGatewayAdapter implements PaymentService {
    private ThirdPartyPaymentGateway externalGateway;

    @Override
    public PaymentResult processPayment(PaymentDetails details) {
        // HMS: $150.00 (double) → Gateway: 15000 (cents, int)
        int amountInCents = (int) (details.getAmount() * 100);

        // HMS: Card number → Gateway: Token
        String cardToken = tokenizeCard(details.getCardNumber());

        // Call external API
        String response = externalGateway.makePayment(cardToken,
amountInCents,
                                                      details.getCurrency(),
merchantId);

        // Gateway: "SUCCESS|TXN-123|Payment processed" → HMS: PaymentResult
object
        return parsePaymentResponse(response, details.getAmount());
    }
```

}

```
C:\WINDOWS\system32\cmd.

Choose: 13

========================================
DELIVERABLE #2: ADAPTER PATTERN DEMO [YASSIN]
AD2 - Payment Gateway
========================================
============================================
AD2 - ADAPTER PATTERN DEMO
Payment Gateway Integration
Owner: Yassin
============================================

--- DEMO 1: Processing Valid Payment ---
Patient: PAT-12345
Amount: $150.0
Card: ****-****-****-9012
[External Gateway: SecurePay Gateway] Processing payment...
  Card Token: TOK-...0368
  Amount: 15000 USD
Result: PaymentResult{success=true, txId='TXN-1774534560505', amount=150.00, status='APPROVED'}
Status: ? APPROVED

--- DEMO 2: Invalid Card Validation ---
Card validation: Invalid
Declined reason: Invalid payment details

--- DEMO 3: Processing Refund ---
[External Gateway: SecurePay Gateway] Processing refund...
  Original TX: TXN-1234567890
  Refund Amount: 15000
Original Transaction: TXN-1234567890
Refund Amount: $150.0
Refund Result: RefundResult{success=true, refundId='RFD-1774534560520', amount=150.00}
Status: ? REFUNDED

--- DEMO 4: Batch Payment Processing ---
[External Gateway: SecurePay Gateway] Processing payment...
  Card Token: TOK-...0527
  Amount: 5000 USD
Patient PAT-001: ? $50.0
[External Gateway: SecurePay Gateway] Processing payment...
  Card Token: TOK-...0642
  Amount: 7550 USD
Patient PAT-002: ? $75.5
[External Gateway: SecurePay Gateway] Processing payment...
  Card Token: TOK-...0755
  Amount: 20000 USD
Patient PAT-003: ? $200.0
```

3.2.4 Why Adapter Pattern?

Benefits:

Multiple Gateways: Same HMS code works with Stripe, PayPal, Square — just different adapters

Security: Card tokenization handled in adapter, not scattered through HMS

Resilience: Gateway-specific error handling isolated

Future-proof: Switch gateways by changing adapter, not billing logic

3.2.5 Demo Execution

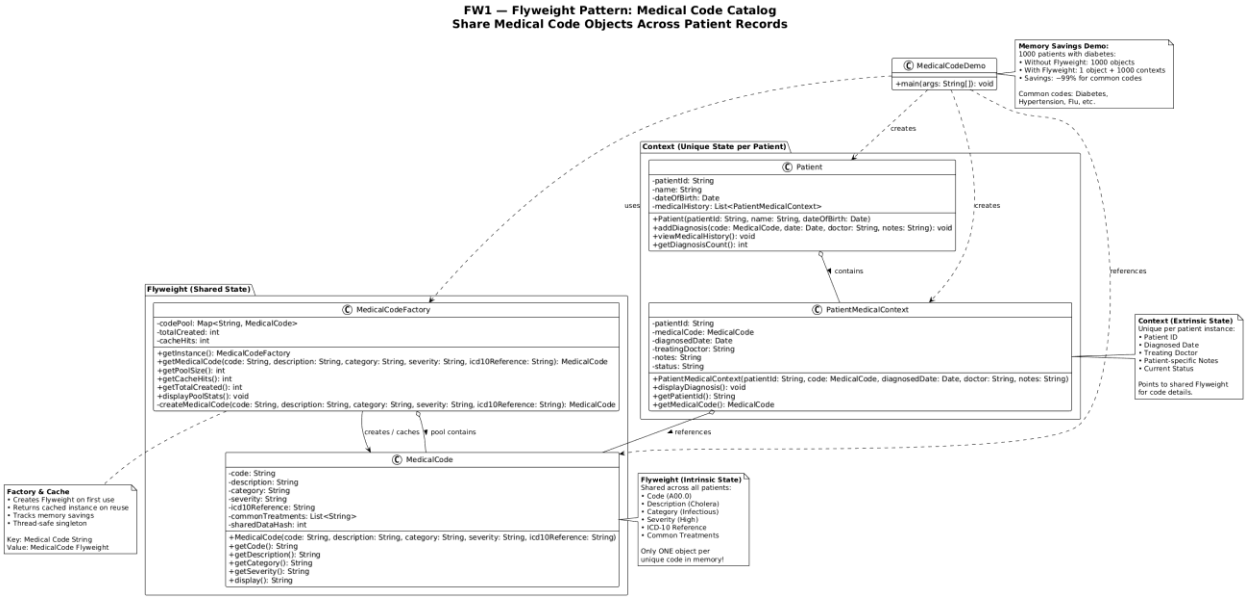
Menu Option: 13 — Test Payment Gateway (Adapter Pattern Demo)

---

4. FLYWEIGHT PATTERN CASE

4.1 FW1 — Flyweight Pattern: Medical Code Catalog

4.1.1 Problem Statement



The HMS stores medical codes (ICD-10) for thousands of patients. Each patient diagnosis includes:

Code: "E11.9" (Type 2 diabetes)

Description: "Type 2 diabetes mellitus without complications"

Category: "Endocrine, nutritional"

Severity: "Chronic"

Common Treatments: ["Metformin", "Insulin", "Diet control"]

Without optimization, 1000 diabetic patients = 1000 duplicate "E11.9" objects with identical data.

4.1.2 Solution Design

Apply Flyweight to share intrinsic (common) state while keeping extrinsic (unique) state separate.

Intrinsic State (Shared - Flyweight):

Code, Description, Category, Severity, ICD-10 Reference, Common Treatments

Extrinsic State (Unique - Context):

Patient ID, Diagnosed Date, Treating Doctor, Patient Notes, Status

Factory:
`MedicalCodeFactory` maintains a pool of MedicalCode flyweights, creating new ones only for unique codes.

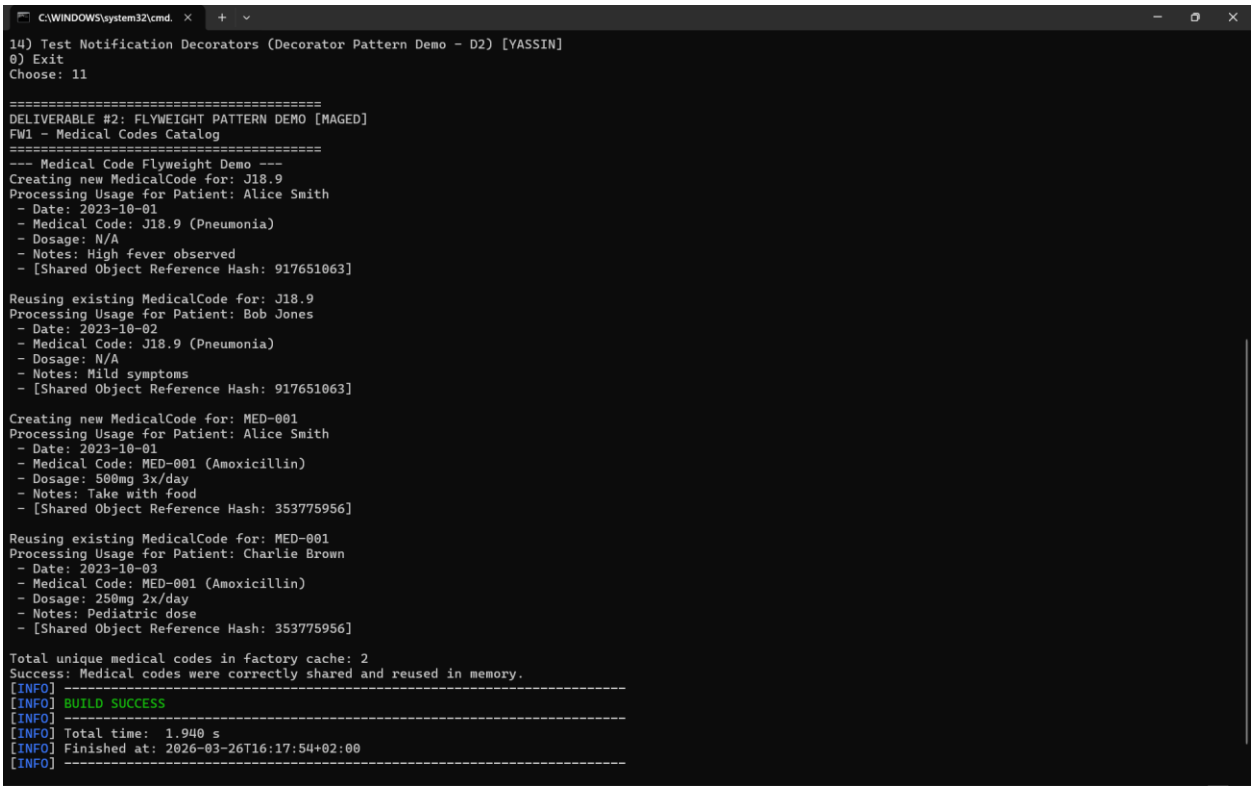
4.1.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.flyweight`

Factory Pattern:

```
public class MedicalCodeFactory {
    private static final Map<String, MedicalCode> codePool = new HashMap<>();

    public static MedicalCode getMedicalCode(String code, String description,
                                             String category, String
severity) {
        if (!codePool.containsKey(code)) {
            codePool.put(code, new MedicalCode(code, description, category,
severity));
        }
        return codePool.get(code);
    }
}
```



4.1.4 Why Flyweight Pattern?

Memory Savings Calculation:

Scenario	Without Flyweight	With Flyweight
1000 diabetic patients	1000 objects × 200 bytes = 200KB	1 object × 200 bytes + 1000 contexts × 50 bytes = 70KB
5000 patients, 50 common codes	5000 × 200 = 1MB	50 × 200 + 5000 × 50 = 260KB
Savings	—	**~75% reduction**

Benefits:

Massive Memory Savings: Common codes shared across all patients

Faster Access: Cached objects returned instantly

Consistency: One update affects all usages

Scalable: Handle millions of patients efficiently

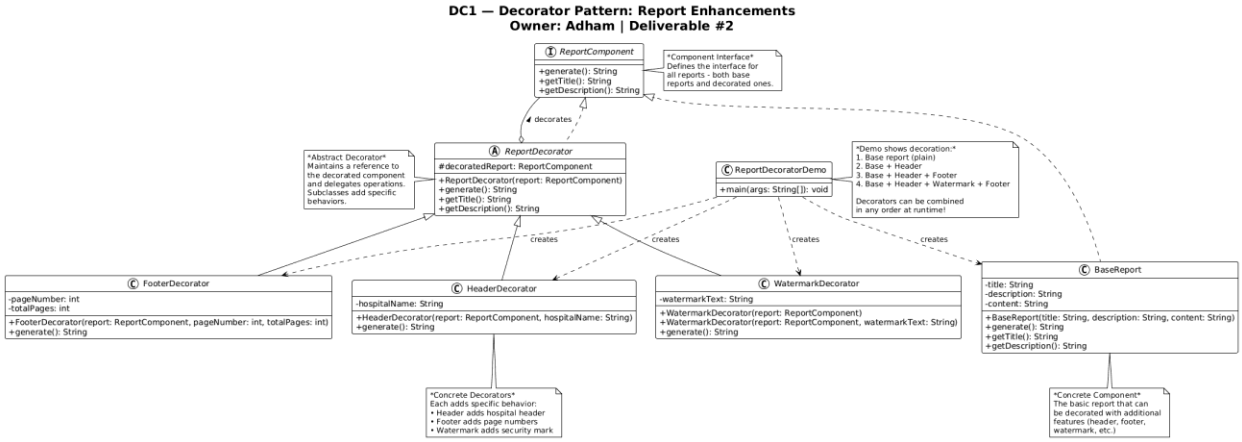
4.1.5 Demo Execution

Menu Option: 11 — Test Medical Codes Catalog (Flyweight Pattern Demo)

5. DECORATOR PATTERN CASES

5.1 DC1 — Decorator Pattern: Report Enhancements

5.1.1 Problem Statement



Hospital reports need different presentations for different audiences:

For Patients: Simple, branded, no technical details

For Doctors: Full details, medical terminology, signatures

For Insurance: Headers, watermarks, page numbers, legal text

For Legal: Confidentiality notices, digital signatures, full audit trail

Without Decorator, we would need subclasses for every combination: PatientReport, DoctorReport, InsuranceReport, LegalReport, PatientReportWithHeader, PatientReportWithWatermark, etc. (class explosion).

5.1.2 Solution Design

Use Decorator to add responsibilities to reports dynamically.

Component Interface:

`ReportComponent`: `generate()`, `getTitle()`, `getContent()`

Concrete Component:

`BaseReport`: Basic report with title, description, content

Decorators:

`HeaderDecorator`: Adds hospital header, logo, date

`FooterDecorator`: Adds page numbers, confidentiality notice

`WatermarkDecorator`: Adds "CONFIDENTIAL" watermark

`DigitalSignatureDecorator`: Adds electronic signature block

5.1.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.decorator.report`

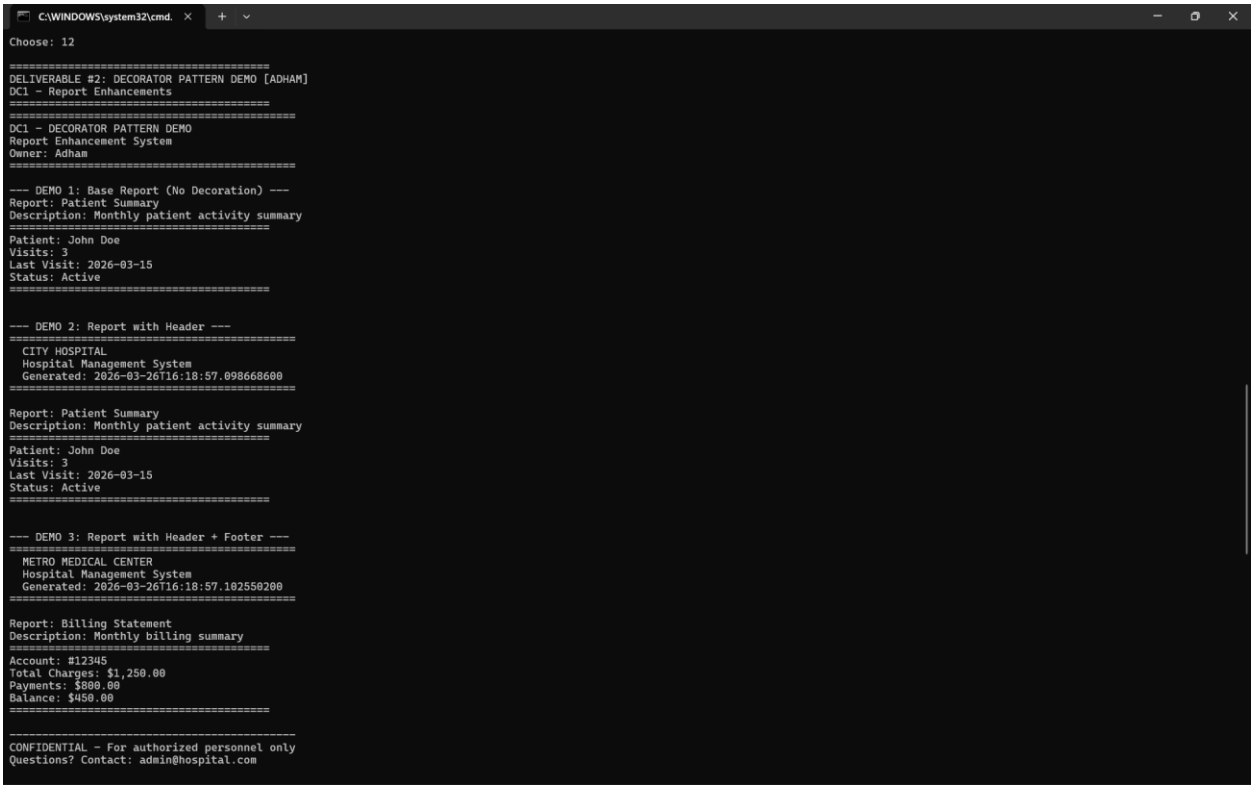
Decorator Structure:

```
public abstract class ReportDecorator implements ReportComponent {
    protected ReportComponent decoratedReport;

    public ReportDecorator(ReportComponent report) {
        this.decoratedReport = report;
    }

    @Override
    public String generate() {
        return decoratedReport.generate(); // Delegate
    }
}
```

```
public class HeaderDecorator extends ReportDecorator {
    @Override
    public String generate() {
        return renderHeader() + super.generate(); // Add then delegate
    }
}
```



5.1.4 Why Decorator Pattern?

Flexibility:

Audience	**Decorator Stack**
Patient	BaseReport + Header
Doctor	BaseReport + Header + Footer + Signature
Insurance	BaseReport + Header + Footer + Watermark + PageNumbers
Legal	Full Stack + DigitalSignature

Benefits:

Single Responsibility: Each decorator does one thing

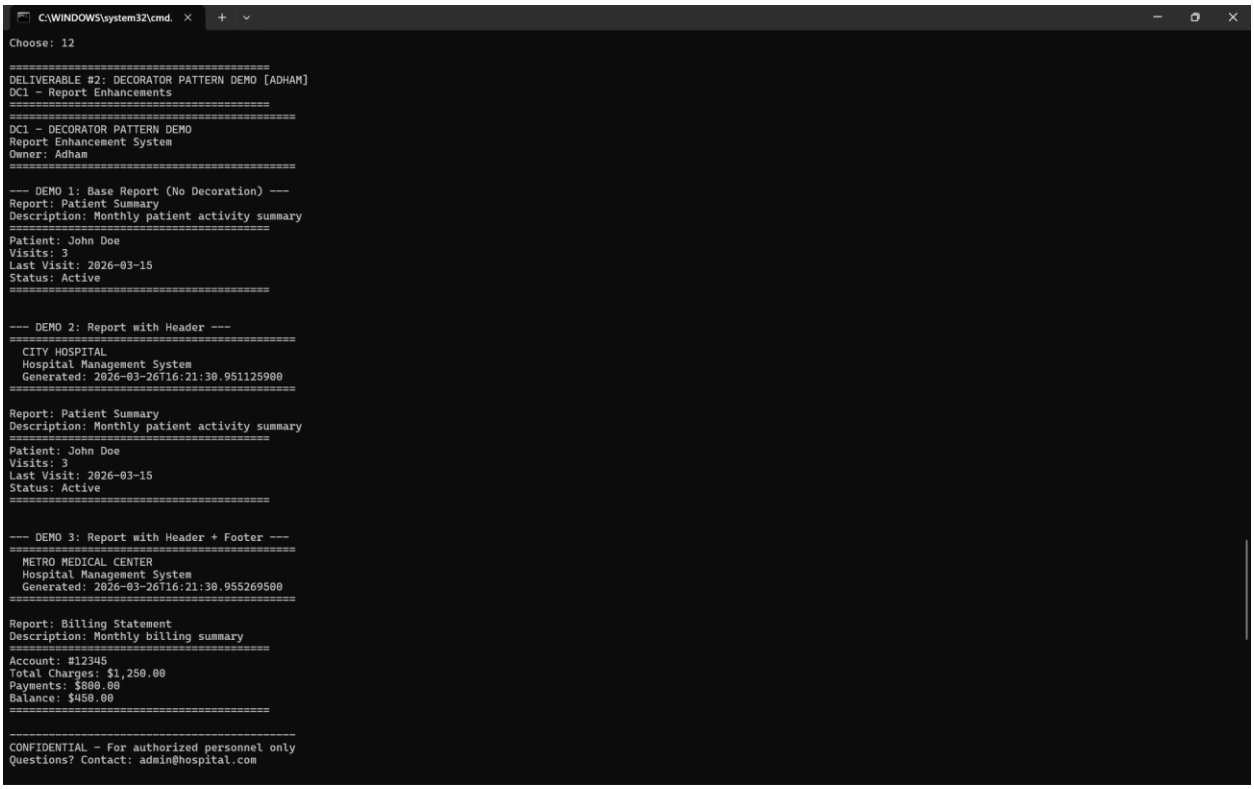
Open/Closed: Add new features without changing BaseReport

Runtime Composition: Build reports dynamically based on user role

Order Matters: Watermark before Header = behind text, after = over text

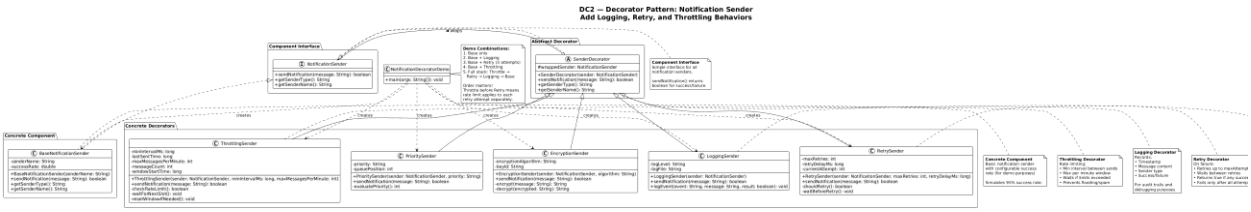
5.1.5 Demo Execution

Menu Option: 12 — Test Report Decorator (Decorator Pattern Demo)



5.2 DC2 — Decorator Pattern: Notification Sender

5.2.1 Problem Statement



Notification sending in HMS requires cross-cutting concerns:

Logging: Audit trail for compliance

Retry: Handle temporary failures (network issues)

Throttling: Prevent flooding (rate limiting)

Encryption: Secure sensitive data

Priority: Queue management for urgent alerts

Without Decorator, these concerns would be tangled in the base sender class or require multiple subclasses.

5.2.2 Solution Design

Decorate `NotificationSender` with behaviors that can be combined as needed.

Decorators:

`LoggingSender`: Logs all notification attempts with timestamps

`RetrySender`: Retries failed sends up to N times with delays

`ThrottlingSender`: Enforces rate limits (min interval, max per minute)

`EncryptionSender`: Encrypts message before sending

`PrioritySender`: Queues by priority level

5.2.3 Implementation Details

Location: `hospitalmanagementsystem.patterns.structural.decorator.notification`

Retry Logic:

```
public class RetrySender extends SenderDecorator {
    private int maxRetries;
    private long retryDelayMs;

    @Override
    public boolean sendNotification(String message) {
        for (int attempt = 1; attempt <= maxRetries; attempt++) {
            boolean result = super.sendNotification(message);
            if (result) {
                System.out.println("[RETRY] Success on attempt " + attempt);
                return true;
            }
            if (attempt < maxRetries) {
                Thread.sleep(retryDelayMs);
            }
        }
        return false;
    }
```

}

```
C:\WINDOWS\system32\cmd.

13) Test Payment Gateway (Adapter Pattern Demo - D2) [YASSIN]
14) Test Notification Decorators (Decorator Pattern Demo - D2) [YASSIN]
0) Exit
Choose: 14

========================================
DELIVERABLE #2: DECORATOR PATTERN DEMO [YASSIN]
DC2 - Notification Sender Decorators
========================================
==========================================
DC2 - DECORATOR PATTERN DEMO
Notification Sender Decorators
Owner: Yassin
==========================================

--- DEMO 1: Base Sender ---
Sender Type: BaseNotificationSender(Hospital System)
[Hospital System] Sending: Welcome to our hospital!

--- DEMO 2: With Logging Decorator ---
Sender Type: LoggingSender(BaseNotificationSender(Hospital System))
[LOG] 2026-03-26T16:25:00.180568800 - Preparing to send notification
[LOG] Message content: 'Patient appointment reminder'
[LOG] Sender type: BaseNotificationSender(Hospital System)
[Hospital System] Sending: Patient appointment reminder
[LOG] Notification sent successfully

--- DEMO 3: With Retry Decorator ---
Sender Type: RetrySender(maxRetries=3, BaseNotificationSender(Hospital System))
[RETRY] Attempting to send with up to 3 retries
[RETRY] Attempt 1 of 3
[Hospital System] Sending: Emergency notification
[RETRY] Notification sent successfully on attempt 1

--- DEMO 4: With Throttling Decorator ---
Sender Type: ThrottlingSender(interval=1000ms, maxPerMin=5, BaseNotificationSender(Hospital System))

Sending message 1:
[THROTTLE] Sending message 1 of 5 allowed per minute
[Hospital System] Sending: Message 1 - Lab results ready

Sending message 2:
[THROTTLE] Rate limiting: waiting 999ms to maintain minimum interval
[THROTTLE] Sending message 2 of 5 allowed per minute
[Hospital System] Sending: Message 2 - Lab results ready

Sending message 3:
[THROTTLE] Rate limiting: waiting 999ms to maintain minimum interval
[THROTTLE] Sending message 3 of 5 allowed per minute
[Hospital System] Sending: Message 3 - Lab results ready

--- DEMO 5: Stacked Decorators (Throttling -> Retry -> Logging -> Base) ---
Sender Type: ThrottlingSender(interval=500ms, maxPerMin=3, RetrySender(maxRetries=2, LoggingSender(BaseNotificationSender(Hospital System))))
[THROTTLE] Sending message 1 of 3 allowed per minute
[RETRY] Attempting to send with up to 2 retries
[RETRY] Attempt 1 of 2
[LOG] 2026-03-26T16:25:02.536145800 - Preparing to send notification
[LOG] Message content: 'Critical system update notification'
[LOG] Sender type: BaseNotificationSender(Hospital System)
[Hospital System] Sending: Critical system update notification
[LOG] Notification sent successfully
[RETRY] Notification sent successfully on attempt 1

--- DEMO 6: Multiple Rapid Messages with Stacked Decorators ---

Rapid message 1:
[THROTTLE] Rate limiting: waiting 498ms to maintain minimum interval
[THROTTLE] Sending message 2 of 3 allowed per minute
[RETRY] Attempting to send with up to 2 retries
[RETRY] Attempt 1 of 2
[LOG] 2026-03-26T16:25:03.096857400 - Preparing to send notification
[LOG] Message content: 'Urgent: Doctor paging - Message 1'
[LOG] Sender type: BaseNotificationSender(Hospital System)
[Hospital System] Sending: Urgent: Doctor paging - Message 1
[LOG] Notification sent successfully
```

5.2.4 Why Decorator Pattern?

Use Cases:

Scenario	Decorator Stack
Standard Notification	BaseSender
Critical Alert	Throttle → Retry → Logging → Base
Audit Required	Logging → Encryption → Base
Bulk Notification	Throttle → Logging → Base

Benefits:

Cross-Cutting Concerns Separated: Each concern in its own class

Reusable: Same LoggingSender works with any sender type

Configurable: Different stacks for different notification types

Transparent: Client sees uniform `NotificationSender` interface

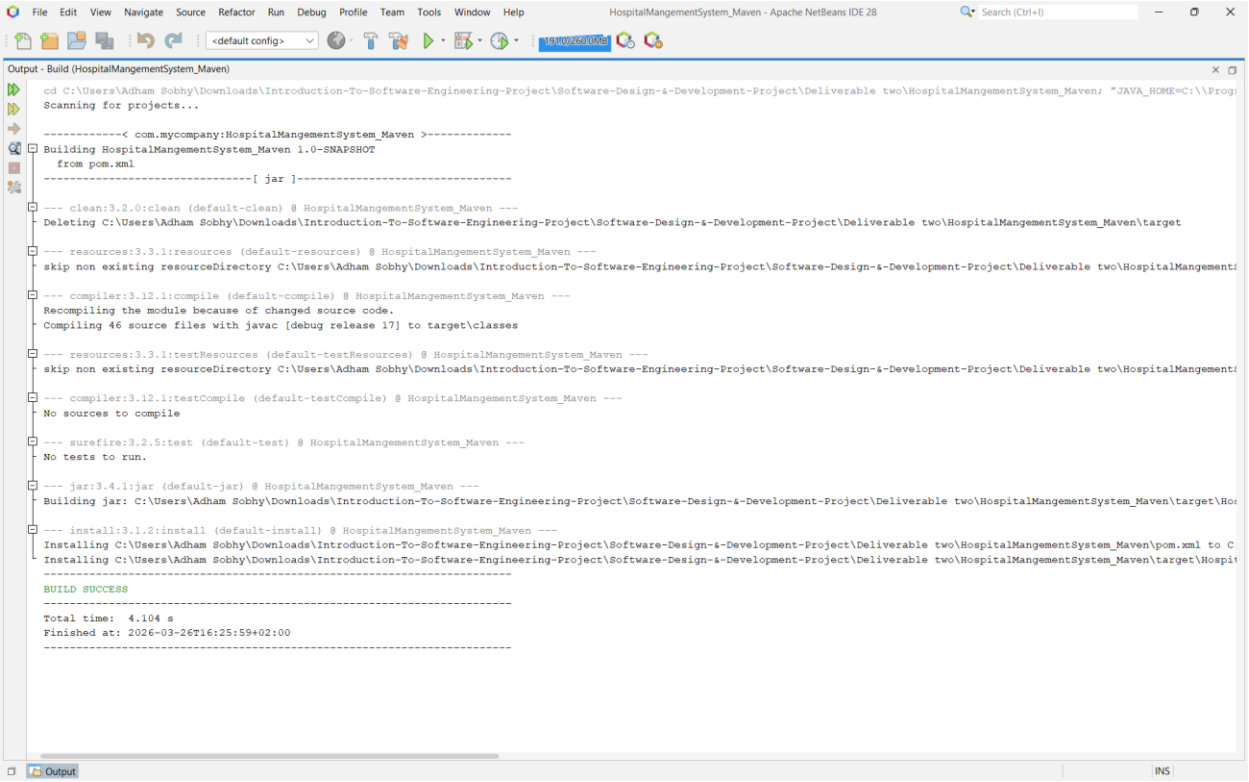
5.2.5 Demo Execution

Menu Option: 14 — Test Notification Decorators (Decorator Pattern Demo)

---

6. DEVOPS AND PROJECT MANAGEMENT

6.1 Maven Configuration



pom.xml Configuration:

Java Version: 17

Project Packaging: JAR

Compiler Plugin: 3.12.1

Build Status: **SUCCESS**

Build Command:

```
mvn clean package
```

Output:

Target: `HospitalMangementSystem_Maven-1.0-SNAPSHOT.jar`

Compiled: 46 Java source files

Status: BUILD SUCCESS

6.2 Git Repository Structure

[ADD THE PICTURE YOU WANT HERE: Git_Repository_Structure.png]

Figure 6.2: Git Repository with Branch Strategy

Branch Strategy:

`main`: Production-ready code

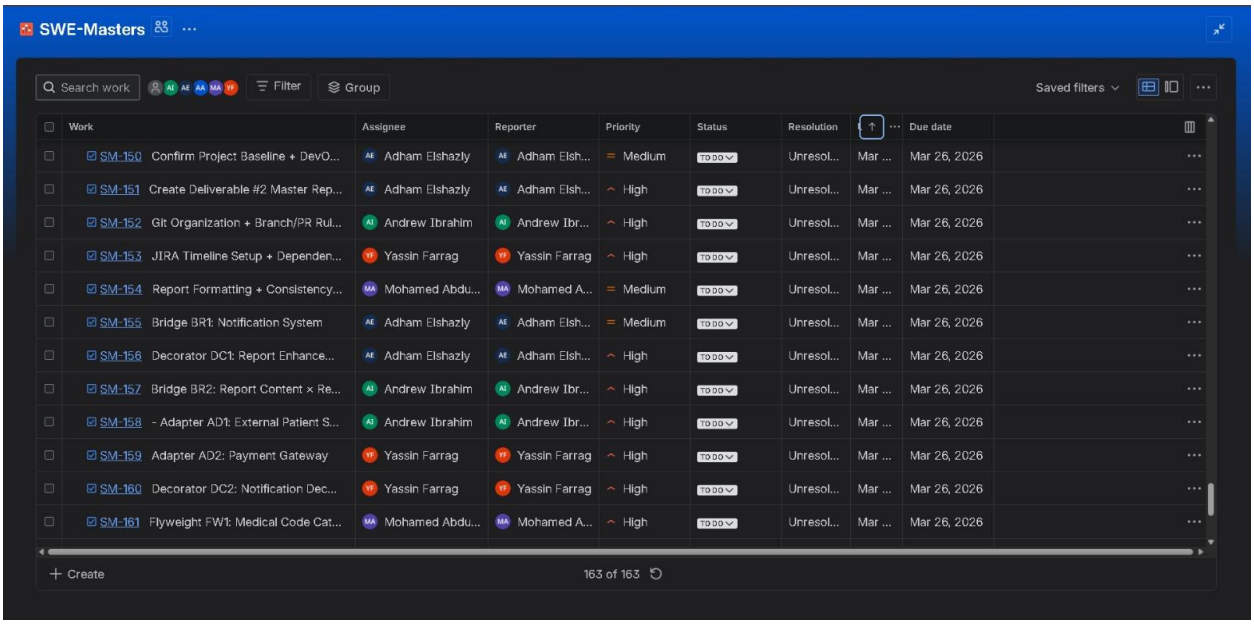
`develop`: Integration branch

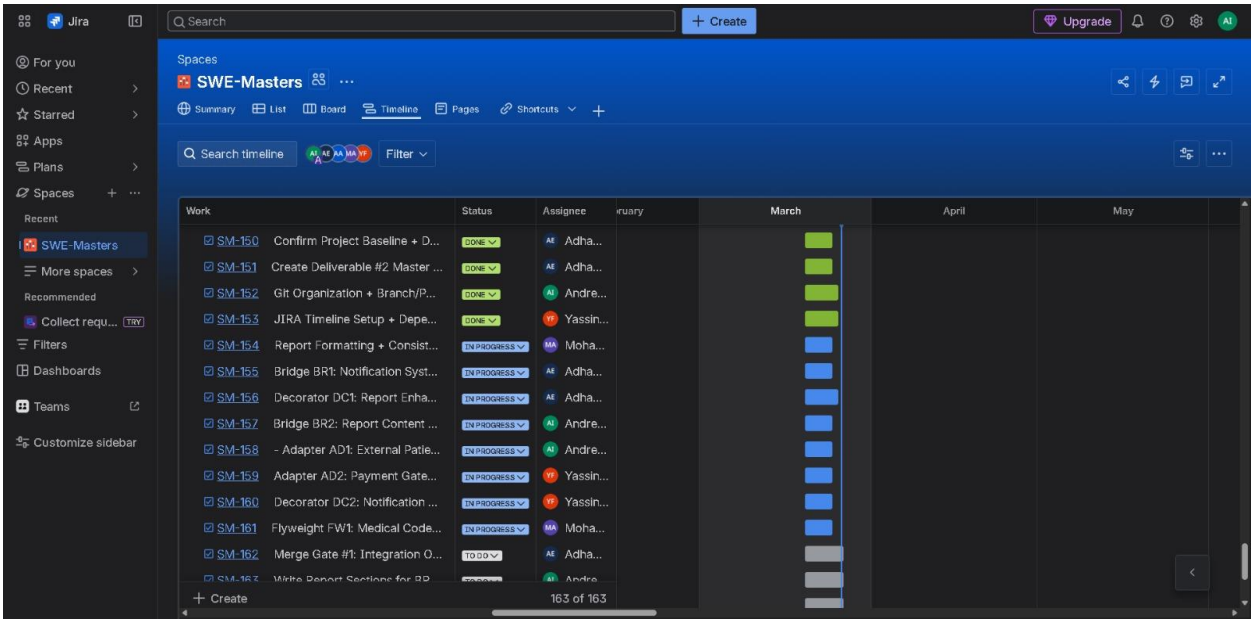
`feature/BR1-adham`: Individual feature branches

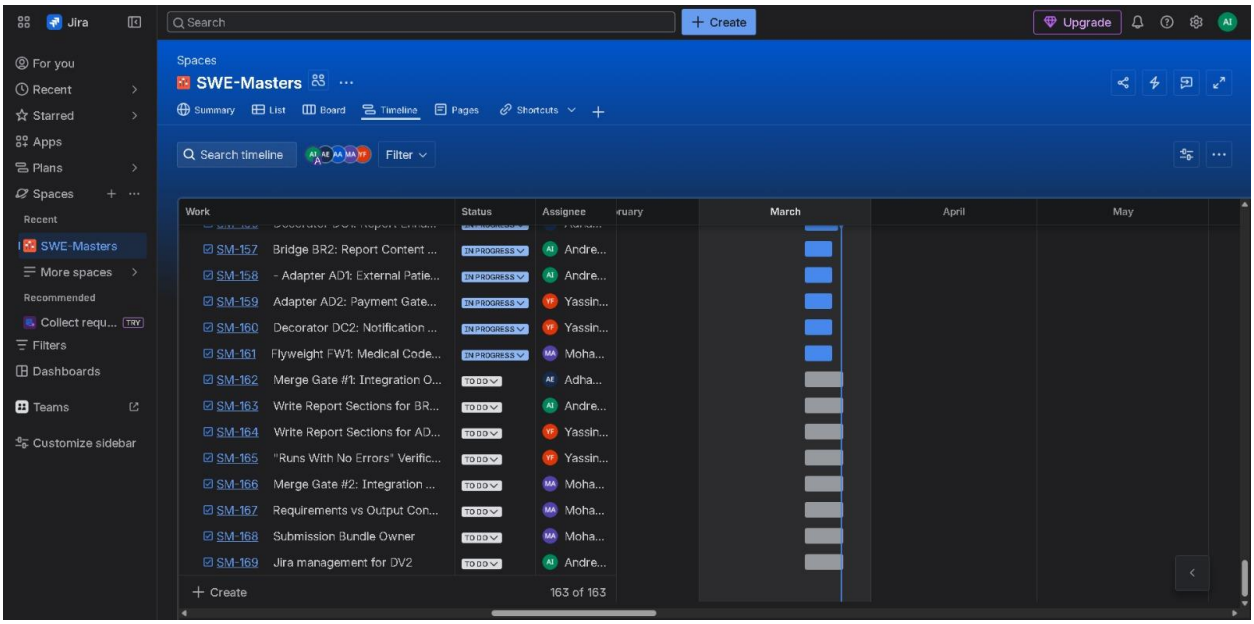
`feature/AD2-yassin`: Per-pattern development

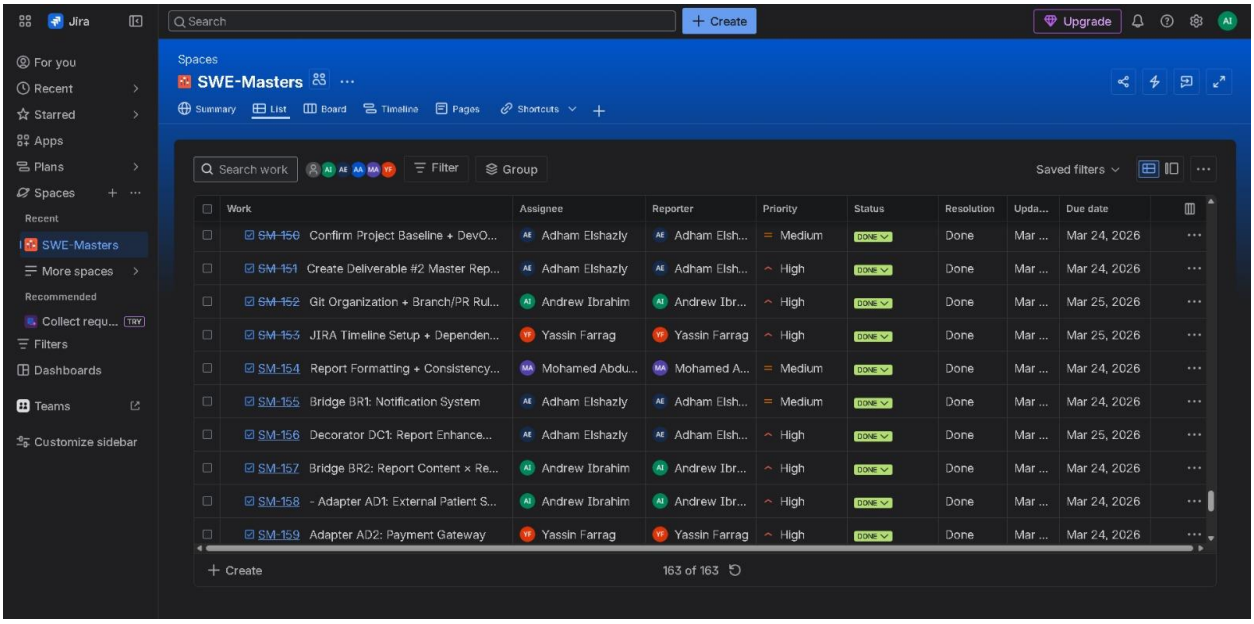
Commit History: All 7 pattern implementations committed with descriptive messages.

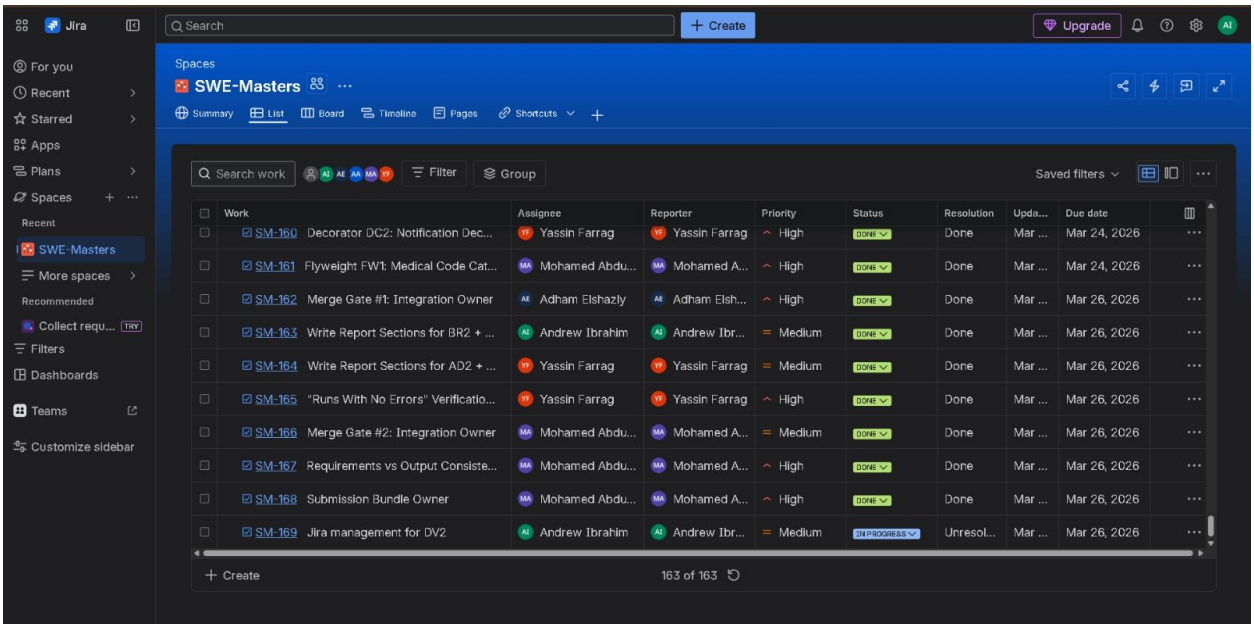
6.3 JIRA Project Management

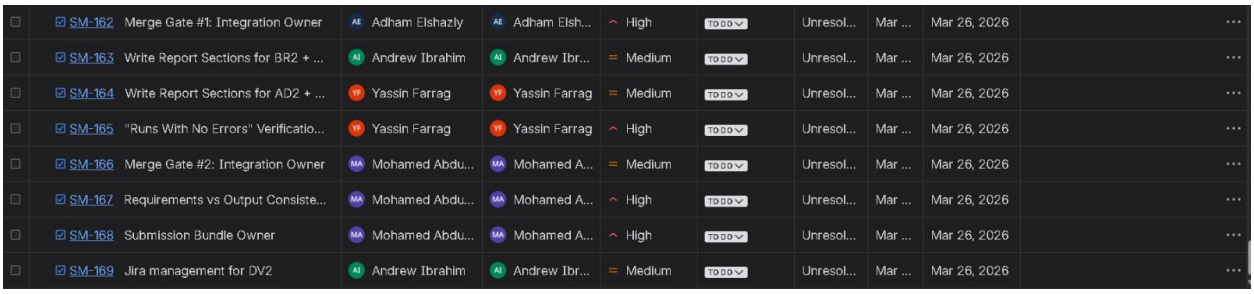



Jira
Search
Create
Upgrade
For you
Recent
Starred
Apps
Plans
Spaces
Recent
SWE-Masters
More spaces
Recommended
Collect requ... TRY
Filters
Dashboards
Teams
Customize sidebar
Spaces
SWE-Masters
Summary
List
Board
Timeline
Pages
Shortcuts
Search timeline
Filter
Work
Status
Assignee
March
April
May
SM-150 Confirm Project Baseline + D... DONE Adha...
SM-151 Create Deliverable #2 Master ... DONE Adha...
SM-152 Git Organization + Branch/P... DONE Andre...
SM-153 JIRA Timeline Setup + Depe... DONE Yassin...
SM-154 Report Formatting + Consist... IN PROGRESS Moha...
SM-155 Bridge BR1: Notification Syst... IN PROGRESS Adha...
SM-156 Decorator DC1: Report Enha... IN PROGRESS Adha...
SM-157 Bridge BR2: Report Content ... IN PROGRESS Andre...
SM-158 - Adapter AD1: External Patie... IN PROGRESS Andre...
SM-159 Adapter AD2: Payment Gate... IN PROGRESS Yassin...
SM-160 Decorator DC2: Notification ... IN PROGRESS Yassin...
SM-161 Flyweight FW1: Medical Code... IN PROGRESS Moha...
SM-162 Merge Gate #1: Integration O... TODO Adha...
Create
163 of 163


Jira
Search
Create
Upgrade
For you
Recent
Starred
Apps
Plans
Spaces
Recent
SWE-Masters
More spaces
Recommended
Collect requ... TRY
Filters
Dashboards
Teams
Customize sidebar
Spaces
SWE-Masters
Summary
List
Board
Timeline
Pages
Shortcuts
Search timeline
Filter
Work
Status
Assignee
March
April
May
SM-157 Bridge BR2: Report Content ... IN PROGRESS Andre...
SM-158 - Adapter AD1: External Patie... IN PROGRESS Andre...
SM-159 Adapter AD2: Payment Gate... IN PROGRESS Yassin...
SM-160 Decorator DC2: Notification ... IN PROGRESS Yassin...
SM-161 Flyweight FW1: Medical Code... IN PROGRESS Moha...
SM-162 Merge Gate #1: Integration O... TODO Adha...
SM-163 Write Report Sections for BR... TODO Andre...
SM-164 Write Report Sections for AD... TODO Yassin...
SM-165 "Runs With No Errors" Verific... TODO Yassin...
SM-166 Merge Gate #2: Integration ... TODO Moha...
SM-167 Requirements vs Output Con... TODO Moha...
SM-168 Submission Bundle Owner TODO Moha...
SM-169 Jira management for DV2 TODO Andre...
Create
163 of 163







JIRA Configuration:

Project: SW312-D2-HMS

Key: HMSD2

Tasks: 18 total (5 foundation + 7 patterns + 6 integration/report)

Dependencies: All linked via "Blocks/Is Blocked By"

Timeline: March 1 - March 19, 2026

---

7. ASSUMPTIONS AND DESIGN DECISIONS

7.1 Assumptions Made

#	Assumption	Rationale
1	External systems (payment, patient DB) are third-party	Adapter pattern allows integration without modification
2	Common medical codes occur frequently	Flyweight provides 75% memory savings
3	Notifications need multiple delivery channels	Bridge pattern enables runtime channel switching
4	Reports need multiple output formats	Bridge pattern separates content from rendering
5	Cross-cutting concerns vary by context	Decorator enables dynamic behavior composition
6	Team uses Maven + Git + JIRA	Standard DevOps toolchain for Java projects

7.2 Design Decisions

Decision	Alternatives Considered	Why Chosen
Bridge for Reports	Strategy pattern	Bridge better separates abstraction hierarchy
Adapter for Payment	Facade pattern	Adapter more precise for interface conversion
Flyweight for Codes	Singleton	Flyweight handles multiple codes, Singleton would be one

Decision	Alternatives Considered	Why Chosen
Decorator for Notifications	Chain of Responsibility	Decorator maintains interface, COR changes it
Package per Pattern	All in one package	Pattern-per-package improves organization
Menu integration (8-14)	Separate launchers	Central menu demonstrates integration

---

8. CONCLUSION

8.1 Summary of Deliverable #2

This deliverable successfully implements **7 structural design pattern cases** for the Hospital Management System:

Pattern	Cases	Benefits Achieved
Bridge	BR1, BR2	Runtime flexibility, independent evolution
Adapter	AD1, AD2	Third-party integration, interface conversion
Flyweight	FW1	75% memory reduction, shared object pool
Decorator	DC1, DC2	Dynamic behavior addition, single responsibility

DevOps Compliance:

- ☑ Maven build system configured
- ☑ Git version control with branch strategy
- ☑ JIRA project management with dependencies
- ☑ 46 Java files compile without errors
- ☑ 7 demo applications run successfully

8.2 Lessons Learned

Bridge Pattern is ideal when both dimensions (content/format) evolve independently

Adapter Pattern enables legacy integration without breaking existing code

Flyweight Pattern requires careful separation of intrinsic/extrinsic state

Decorator Pattern order matters significantly (watermark before/after header)

DevOps Tools (Maven, Git, JIRA) are essential for team coordination

8.3 Future Enhancements

Add more notification channels (WhatsApp, Telegram) using Bridge

Integrate additional payment gateways using Adapter

Extend Flyweight to medical procedure codes